

Journal de bord de stage

Aria Project - Suivi hebdomadaire des activités

Étudiant	Ghazi Mabrouki
Université	ESPRIT
Formation	ArcTic
Entreprise / Projet	Aria Project
Période	12 janvier 2026 - 10 avril 2026

Objet du document. Ce journal de bord présente l'évolution du stage semaine par semaine, les compétences développées, les difficultés rencontrées et les résultats obtenus dans la mise en place d'un SOC automatisé autour du projet Aria.

Vue d'ensemble du stage

Contexte	Stage orienté cybersécurité, centralisation des logs et automatisation de la réponse aux incidents.
Objectif principal	Mettre en place et faire évoluer un environnement Aria connecté à Elasticsearch, Wazuh, Suricata, Falco et Redis.
Résultat global	Un pipeline fonctionnel, structuré et enrichi par des mécanismes de filtrage, de déduplication et d'automatisation.

Principales phases du stage

- Phase 1 : préparation de l'environnement serveur, réseau et sécurité de base.
- Phase 2 : déploiement de la stack d'observabilité et de détection (Elastic, Kibana, Wazuh, Suricata, Falco, Filebeat).
- Phase 3 : intégration d'OpenSOAR, du backend et des mécanismes d'automatisation.
- Phase 4 : amélioration continue avec filtrage, déduplication, génération de playbooks et watcher d'incidents.

Semaine 1 - 12 au 18 janvier 2026

Mission principale : Configuration de l'environnement cloud et installation du système de base

Activités réalisées

- Mise en place du serveur cloud (193.95.30.97).
- Installation du système d'exploitation Linux (Ubuntu/Debian).
- Configuration réseau : IP statique, DNS et passerelle.
- Installation des paquets essentiels : curl, wget, git, nano et htop.
- Configuration SSH et clés d'authentification.
- Mise à jour du système et renforcement de la sécurité de base (pare-feu, Fail2ban).

Apprentissages

- Administration avancée d'un serveur Linux.
- Configuration réseau de base en environnement cloud.
- Sécurisation Linux : UFW et durcissement SSH.

- Automatisation simple via commandes système et scripts Bash.

Difficultés rencontrées

- Première expérience concrète avec une infrastructure cloud.
- Compréhension du fonctionnement réseau dans le cloud (NAT, routage).

Solutions apportées

- Création d'une documentation personnelle des commandes utiles.
- Appui sur des guides techniques et tutoriels fiables.
- Tests en environnement isolé avant la mise en production.

Réflexion

La configuration initiale constitue la base de tout le projet. Un serveur mal préparé peut compromettre les étapes suivantes ; le temps investi au démarrage est donc rapidement rentabilisé.

Semaine 2 - 19 au 25 janvier 2026

Mission principale : Installation et configuration d'Elasticsearch et Kibana

Activités réalisées

- Installation d'Elasticsearch 8.x via APT.
- Configuration du fichier elasticsearch.yml.
- Mise en place de la sécurité native (X-Pack / Security).
- Ajustement de la mémoire JVM (heap size).
- Installation de Kibana.
- Configuration d'un reverse proxy Nginx pour Kibana.
- Génération de certificats SSL auto-signés.
- Tests de connectivité et validation du cluster.

Apprentissages

- Architecture Elasticsearch : indices, shards et replicas.
- Utilisation de l'API REST Elasticsearch pour la recherche et l'agrégation.
- Configuration de Kibana et création de dashboards.
- Notions de mapping et d'analyse de texte.

Difficultés rencontrées

- Taille mémoire JVM insuffisante, causant des erreurs de type OutOfMemory.
- Certificats SSL non reconnus par le navigateur.
- Problèmes de droits sur les répertoires de données.

Solutions apportées

- Import du certificat auto-signé dans le trust store du navigateur.
- Correction des permissions avec l'utilisateur elasticsearch.

Réflexion

Elasticsearch représente le cœur de la stack SIEM. Sa stabilité et ses performances dépendent fortement de la qualité de la configuration initiale.

Semaine 3 - 26 janvier au 1er février 2026

Mission principale : Installation et configuration de Wazuh

Activités réalisées

- Installation du Wazuh Manager via script automatisé.
- Configuration du fichier ossec.conf
- Définition de règles de détection personnalisées.
- Installation d'agents sur les machines cibles.
- Enregistrement des agents auprès du manager.
- Validation par génération volontaire de faux positifs pour tester les alertes.

Apprentissages

- Fonctionnement de Wazuh comme HIDS/HIPS open source.
- Syntaxe des règles Wazuh : decoders et rules.
- Gestion, supervision et enrôlement des agents.
- Niveaux d'alerte, catégories et groupes d'événements.
- Intégration avec Elasticsearch via le composant Wazuh Indexer.

Difficultés rencontrées

- Incompatibilités entre certains agents et versions Linux.
- Règles initialement trop sensibles générant beaucoup de faux positifs.
- Problèmes de communication agent-manager liés au pare-feu.

Solutions apportées

- Utilisation de la dernière version agent compatible avec l'environnement.
- Ajustement des niveaux de criticité des règles.
- Ouverture des ports nécessaires : 1514, 1515 et 1516.

Réflexion

Wazuh apporte une excellente couche de supervision hôte. Son intégration avec

Elastic facilite la corrélation, mais un travail de tuning reste indispensable.

Semaine 4 - 2 au 8 février 2026

Mission principale : Installation de Suricata et configuration IDS/IPS

Activités réalisées

- Installation de Suricata depuis les dépôts ou les sources.
- Configuration du fichier suricata.yaml.
- Téléchargement et activation des règles ET Open.
- Configuration du mode IDS.
- Activation de la sortie Eve.json pour les logs structurés.
- Intégration avec Elasticsearch via Filebeat.
- Tests sur des scénarios simulés (nmap, hping3).

Apprentissages

- Principe de fonctionnement des IDS/IPS.
- Syntaxe des règles Suricata : signatures, actions et mots-clés.
- Différences entre modes IDS passif et IPS actif.
- Analyse de trafic et compréhension des protocoles réseau.
- Structure du format Eve.json.

Difficultés rencontrées

- Volume de logs très élevé en période de pointe.
- Faux positifs liés aux règles par défaut.
- Contraintes de performance sur port mirroring.

Solutions apportées

- Mise en place d'un filtrage par seuils.
- Désactivation des catégories de règles les plus bruyantes.
- Utilisation de filtres BPF pour réduire le trafic analysé.

Réflexion

Suricata est un outil très performant pour la détection réseau. La qualité des résultats dépend surtout de la bonne sélection des règles actives afin d'éviter le bruit.

Semaine 5 - 9 au 15 février 2026

Mission principale : Installation de Falco et sécurité runtime des conteneurs

Activités réalisées

- Installation de Falco.
- Configuration des règles Falco.
- Ajout de règles adaptées à l'audit Kubernetes.
- Configuration de l'export vers Elasticsearch via falcosidekick / Filebeat.
- Tests de détection avec simulation d'activités suspectes dans les conteneurs.
- Mise en relation avec les alertes Suricata et Wazuh.

Apprentissages

- Sécurité runtime des conteneurs.
- Analyse comportementale basée sur les syscalls.
- Structure des règles Falco : conditions, outputs et priorités.
- Différentes sources de télémétrie : syscall, audit Kubernetes et cloud.
- Construction d'une politique de sécurité orientée conteneurs.

Difficultés rencontrées

- Impact potentiel sur les performances en production.
- Volume important d'événements système.
- Règles trop génériques générant des faux positifs.

Solutions apportées

- Priorisation des règles pour ignorer les alertes de faible importance.
- Réutilisation des règles officielles comme base de travail.
- Personnalisation progressive selon l'environnement réel.

Réflexion

Falco complète la visibilité du SOC côté host et runtime. Avec Suricata pour le réseau et Falco pour les comportements système, la couverture devient beaucoup plus riche.

Semaine 6 - 16 au 22 février 2026

Mission principale : Configuration de Filebeat et centralisation des logs

Activités réalisées

- Installation de Filebeat sur les différents composants.
- Activation des modules Filebeat : Wazuh, Suricata et System.

- Mise en place de Logstash pour la transformation et l'enrichissement.
- Création de pipelines de parsing personnalisés.
- Configuration de la sortie vers Elasticsearch.
- Validation de la réception des différents types de logs.

Apprentissages

- Architecture ELK : Beats -> Logstash -> Elasticsearch.
- Différence entre modules Filebeat et configuration manuelle.
- Enrichissement par GeoIP, hostname .

Difficultés rencontrées

- Hétérogénéité des formats de logs entre les sources.
- Conflits de mapping et collisions de types dans Elasticsearch.
- Rate limiting côté Elasticsearch lors des pics de charge.

Solutions apportées

- Utilisation prioritaire des modules Beats préconfigurés.
- Création de templates d'indexation adaptés.
- Mise en place de mécanismes de throttling et de backpressure.

Réflexion

La centralisation des logs est un prérequis fondamental pour tout SOC moderne. Sans elle, il devient impossible de corréler efficacement les événements issus de sources hétérogènes.

Semaine 7 - 23 février au 1er mars 2026

Mission principale : Installation et configuration de Redis

Activités réalisées

- Installation du serveur Redis.
- Configuration du fichier redis.conf
- Configuration éventuelle de la réplication.
- Intégration de Redis comme cache pour le backend.

Apprentissages

- Redis comme cache et magasin clé-valeur.
- Transactions, pipelines et optimisation des accès.

Difficultés rencontrées

- Gestion de la mémoire lorsque le volume augmente.
- Ralentissements liés à la persistance sur des charges d'écriture importantes.
- Limitation des connexions simultanées.

Solutions apportées

- Configuration de maxmemory et des politiques d'éviction.
- Utilisation d'un connection pooling côté application.

Réflexion

Redis s'est révélé indispensable dans le pipeline de traitement des alertes. Il permet d'améliorer les performances grâce au cache et de faciliter les mécanismes de déduplication.

Semaine 8 - 2 au 8 mars 2026

Mission principale : Installation d'OpenSOAR et configuration initiale

Activités réalisées

- Installation d'OpenSOAR côté backend et frontend.
- Configuration de la base SQLite.
- Paramétrage de l'authentification administrateur.
- Configuration des paramètres globaux.
- Connexion avec Elasticsearch.
- Vérification de la connectivité de l'API.

Apprentissages

- Architecture applicative d'OpenSOAR : FastAPI .
- Structure des données incidents / alertes.
- Principaux endpoints de l'API REST.
- Workflow de gestion des incidents et des investigations.

Difficultés rencontrées

- Nombre important de variables d'environnement à configurer.

- Conflits de ports avec d'autres services déjà présents.
- Problèmes de permissions sur certains fichiers.

Solutions apportées

- Centralisation de la configuration dans un fichier .env.
- Nettoyage des ports en conflit avant déploiement.
- Correction des droits système sur les fichiers concernés.

Réflexion

OpenSOAR constitue l'interface centrale du SOC. Sa prise en main demande une bonne compréhension du cycle de vie des incidents pour adapter correctement les workflows.

Semaine 9 - 9 au 15 mars 2026

Mission principale : Initialisation et configuration du backend OpenSOAR

Activités réalisées

- Création d'un environnement virtuel Python.
- Installation des dépendances via requirements.txt.
- Configuration du fichier .env avec les paramètres nécessaires.
- Tests de connexion à Elasticsearch.
- Tests de connexion à Redis.
- Tests de connexion à l'API OpenSOAR.
- Lancement du backend en mode développement.

Apprentissages

- Structure d'un projet Python / FastAPI.
- Utilisation de l'asynchrone avec async/await pour les échanges réseau.
- Gestion centralisée de la configuration par variables d'environnement.
- Mise en place de tests d'intégration avec des services externes.

Difficultés rencontrées

- Résolution de l'ensemble des dépendances Python.
- Incompatibilités possibles avec Python 3.12.
- Gestion de dépendances transitives parfois fragiles.

Solutions apportées

- Création d'un virtualenv dédié au projet.
- Utilisation de versions figées dans requirements.txt.

- Validation progressive des imports et dépendances.

Réflexion

Le backend représente le moteur de l'automatisation. Bien comprendre sa structure est indispensable pour étendre les fonctionnalités et fiabiliser les traitements.

Semaine 10 - 16 au 22 mars 2026

Mission principale : Implémentation du système de filtrage des alertes (Sigma Rules)

Activités réalisées

- Création de 10 fichiers de règles Sigma pour filtrer le bruit.
- Configuration du système de filtrage dans le poller.
- Tests avec les alertes réelles provenant de chaque source.
- Ajustement progressif des seuils de filtrage.
- Validation du bon passage des vraies alertes.

Apprentissages

- Syntaxe et structure des règles Sigma.
- Logique de détection basée sur des patterns.
- Différence entre faux positifs et bruit système.
- Méthodologie de test et de validation de règles.

Difficultés rencontrées

- Trouver le bon équilibre entre réduction du bruit et conservation des alertes utiles.
- Difficulté à distinguer certains patterns légitimes de comportements anormaux.

Solutions apportées

- Tests empiriques sur des données réelles.
- Approche itérative avec ajustements progressifs.

Réflexion

Le bruit constitue l'un des principaux freins à la productivité d'un SOC. Un filtrage bien conçu améliore fortement la lisibilité et la pertinence des alertes.

Semaine 11 - 23 au 29 mars 2026

Mission principale : Système de déduplication et groupement des alertes

Act vités réalisées

- Mise en place d'un système de groupement d'alertes similaires.
- Comptage des occurrences pour réduire le bruit.
- Configuration des TTL pour le nettoyage automatique.
- Tests de performance sur des volumes importants.

Apprent ssages

- Algorithmes de déduplication en temps réel.
- Optimisation des opérations Redis pour la performance.

Difficultés rencontrées

- Consommation mémoire importante lors de fortes répétitions.
- Temps de traitement plus long sur des volumes élevés.

Solut ons apportées

- Traitement par lots pour limiter la charge.

Réffexion

La déduplication évite de traiter plusieurs fois une même alerte. Le groupement apporte en plus une vision plus claire de la fréquence et de la récurrence des incidents.

Semaine 12 - 30 mars au 5 avril 2026

Mission principale : Implémentation du moteur IA pour la génération de playbooks Ansible

Act vités réalisées

- Configuration du LLM (Ollama avec le modèle qwen3:8b).
- Création du moteur IA dans response/ai_engine.py.
- Intégration avec l'API du modèle.
- Implémentation de la génération automatique de playbooks YAML.
- Validation de la syntaxe via `yaml.safe_load()`.
- Gestion des timeouts entre 60 et 120 secondes.

Apprent ssages

- Intégration d'un LLM via API REST.
- Prompt engineering pour la génération de code.

- Validation et sécurisation des sorties générées.
- Gestion des erreurs, retries et timeouts.
- Comparaison pratique entre plusieurs approches et modèles.

Difficultés rencontrées

- Qualité variable des playbooks générés.
- Temps de réponse du LLM parfois long.
- Présence ponctuelle d'erreurs de syntaxe.

Solutions apportées

- Validation stricte des réponses avec `yaml.safe_load()`.
- Mise en place de timeouts adaptés à la complexité.
- Amélioration des prompts pour guider la génération.

Réflexion

L'IA accélère la génération de playbooks, mais elle ne remplace pas l'expertise humaine. Une validation humaine reste nécessaire avant toute exécution sensible.

Semaine 13 - 6 avril 2026

Mission principale : Implémentation du watcher pour la détection des incidents

Activités réalisées

- Création du fichier `response/watcher.py`.
- Implémentation d'une boucle de polling toutes les 60 secondes.
- Appels API vers OpenSOAR pour récupérer les incidents.
- Logique de création automatique des investigations.
- Système de liaison entre alertes et incidents.
- Gestion des états : `pending` -> `awaiting_approval` -> `approved`.

Apprentissages

- Utilisation détaillée de l'API REST OpenSOAR.
- Manipulation de SQLAlchemy en mode asynchrone.
- Gestion des cycles de polling, retry et transformation des données.

Difficultés rencontrées

- Présence de certains incidents sans alertes associées (`created_from_api`).
- Extraction du `target_host` parfois vide.

Solutions apportées

- Ajout d'un logging plus détaillé pour le débogage.

- Mise en place d'un fallback vers `ansible_remote_host`.

Réflexion

Le watcher joue un rôle clé dans le déclenchement de l'automatisation. Sans lui, les incidents détectés ne peuvent pas être pris en charge de manière fluide par le pipeline.

Compétences acquises durant le stage

Compétence	Niveau	Description
Linux Server	★★★★☆	Installation, configuration et sécurisation
Elasticsearch	★★★☆☆	Installation, requêtes et optimisation
Kibana	★★★☆☆	Dashboards et visualisation
Wazuh	★★★☆☆	HIDS, agents et règles
Suricata	★★★☆☆	IDS/IPS et analyse réseau
Falco	★★☆☆☆	Sécurité runtime des conteneurs
Python / FastAPI	★★★☆☆	Backend, API REST et asynchrone
Redis	★★★☆☆	Cache et déduplication
Filebeat / ELK	★★★☆☆	Centralisation et transport des logs
Docker	★★★☆☆	Conteneurisation
Git	★★★☆☆	Gestion de versions
Ansible	★★☆☆☆	Automatisation et playbooks
LLM / IA	★★☆☆☆	Intégration IA pour la génération

Statistiques du projet

Composant	Statut	Détails
Elasticsearch	Opérationnel	Cluster fonctionnel
Kibana	Configuré	Dashboard prêt à l'emploi
Wazuh	Déployé	1 agent intégré
Suricata	Actif	Mode IDS opérationnel
Falco	Installé	Configuration validée
Filebeat	Actif	4 modules activés
Redis	Opérationnel	Cache fonctionnel
OpenSOAR	Déployé	Backend et frontend disponibles
Backend Poller	Actif	4 sources : Wazuh, Falco, Suricata, Filebeat
Sigma Rules	Actif	10 règles de filtrage
AI Engine	Configuré	LLM intégré
Investigations	123	Total créées
Playbooks	24	Exécutés

Problèmes rencontrés et solutions

Problème 1

Description	101 investigations bloquées en "awaiting_approval"
Cause	Absence de processus d'approbation automatisé.
Solution	Implémentation de <code>_execute_approved_investigations()</code> dans <code>watcher.py</code> .

Problème 2

Description	Vérification non déclenchée après échec du playbook
Cause	La vérification ne s'exécutait que lorsque <code>exit_code = 0</code> .
Solution	Déclenchement systématique ajouté dans <code>ansible_exec.py</code> .

Problème 3

Description	Manque de visibilité sur les investigations bloquées
Cause	Aucune notification n'était prévue.
Solution	Ajout de <code>_check_stuck_investigations()</code> avec alertes Slack.

Problème 4

Description	Erreurs masquées par des <code>bare except</code>
Cause	Utilisation d'exceptions trop larges.
Solution	Remplacement par des exceptions spécifiques.

Prochaines étapes

- ☐ Interface d'approbation via Slack.
- ☐ Mise en place d'un système de sauvegarde automatique.
- ☐ Mécanisme d'auto-approbation pour les cas de basse priorité.
- ☐ Ajout de tests unitaires plus complets.
- ☐ Système de supervision des performances avec Telegraf, Prometheus et suivi des processus.

Réflexion générale

Ce stage m'a permis de comprendre le fonctionnement global d'un SOC automatisé, depuis la collecte des logs jusqu'à la réponse aux incidents. Le projet Aria constitue un cadre concret pour mettre en pratique la centralisation, la corrélation, le filtrage et l'automatisation. L'expérience montre toutefois que, même avec une chaîne technique fonctionnelle, le facteur humain reste essentiel pour valider les actions

sensibles et superviser les cas complexes.